Project Design Phase-II
Solution Requirements (Functional & Non-functional)

Date	18 June 2026
Team ID	
Project Name	ShopEZ-E-commerce-Application
Maximum Marks	4 Marks

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User Security & Authentication	Registration through Form (Role selection: User/Admin) Secure login with JWT-based session management Password encryption
FR-2	User Confirmation	Confirmation via Email Confirmation via OTP
FR-3	Portfolio Management	Calculate and display real-time profit/loss metrics
FR-4	Admin Moderation & Controls	Dashboard to view total active users and platform metrics.

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

FR No.	Non-Functional Requirement	Description
NFR-1	**Usability**	The platform must feature a responsive, data-driven user interface accessible across desktop and mobile devices, utilizing clear visual cues (e.g., green/red color coding) for profit and loss metrics.

NFR-2	**Security**	The system must enforce strict role-based access control, utilize CORS to secure API endpoints, and ensure all sensitive user data and transaction logs are protected within the database.
NFR-3	**Reliability**	The application must guarantee accurate, real-time balance updates and precise logging of trade details (quantity, price, timestamp) without data loss during concurrent simulated trading.
NFR-4	**Performance**	The backend must utilize optimized MongoDB queries (e.g., indexing on stock symbols), and the React frontend must optimize rendering and API calls to prevent UI lag during live market auto-refreshing.
NFR-5	**Availability**	The system should be deployed on robust cloud infrastructure to ensure consistent uptime, allowing users continuous access to their virtual portfolios and live market data during standard trading hours.
NFR-6	**Scalability**	The Express.js backend controllers and services must be refactored for clean separation of concerns, ensuring the architecture can handle an increasing volume of users and simulated transaction data.